DS 오답코드 정리

20190252 이윤주

Data Structure

Data structure
: 저장 방식과
그것에 접근, 변경
하는 것

-Data Structure란?

Storage method 와 그것을 access, 혹은 modify 할 때 쓰이는 algorithm 을 말함.

-Abstract Data Type(ADT)
: 함수의 특징과 함수의 동작 방식에 대해 서술한 것. 구체적 구현은 나타나 있지 않음

Program을 ADT형태로 작성하는 것이 중요한 첫 번째 단계임.
.h 파일의 앞 부분에 적어 줌.

Operation – 1. Precondition 2. Postcondition 3. return value

Taxonomy of Errors

Syntax
System
& validity

System Error => syntax Error

Run-Time Error => mainly from misuse of memory and divide by zero : 배열 인덱스 범위 초과 호출 등

Validity Error => Logic Error : 잘못 생각한 경우

Verification Errors: System requirements analysis failure : 안에서 기능이 빠져 있는 경우 등

Maintenance Errors

Two techniques for Unit Testing

1. **Drivers – 입력, 실행, 출력을 test함.**
2. **Stubs – 마치 실행된 것처럼 보이게 만듦.**

Max SelectionSort

-잘못 짠 코드 부분 : maxSelect 함수

```
while(currentPos<n)
{
   if(a[currentPos]>a[maxPos])
     {
       maxPos=currentPos;
       currentPos+=1;
     }
```

-무슨 생각으로? If문이 실행될 때 큰 값을 가지고 있는 배열의 인덱스를 max 인덱스에 넣고 그 다음 currentPos 인덱스를 줄여 나가는 건데 이게 if문에서 같이 실행된다고 생각함.

정답 코드 currentPos <n

```
while(currentPos<n)
{
 if(a[currentPos]>a[maxPos])
  {
    maxPos=currentPos;
  }
currentPos++;
}
```

생각해보면 졸라 간단한 게 if 문이 실행되지 않아도 currentPos의 값은 증가시켜야 하기 때문임.

Min SelectionSort

-잘못 짠 코드 부분 : SelectionSort 함수

```
void SelectionSort(int a[],int n)
{
  int minPos;
  int last(n-1);
  while(last>0)
  {
    minPos = minPos(n,last);
    swapElements(a,minPos,last);
  }
  last--;
```

정답 코드

```
void SelectionSort(int a[],int n)
{
  int minPos;
  int last(n-1);
  while(last>0)
  {
    minPos = minPos(n,last);
    swapElements(a,minPos,last);
    last--;
  }
}
}
```

List Array

-잘못 짠 코드 부분 : next 함수

```
bool List::next(ListElementType& elem)
{
  if (numberOfElements>maxListSize)
  {
     return false;
  }
  else
  {
     currentPosition = numberOfElements;
     elem = listArray[currentPosition];
```

정답 코드

```
bool List::next(ListElementType& elem)
{
  assert(currentPosition >= 0);
  if (currentPosition >= numberOfElements - 1)
  {
     return false;
  }
  else
  {
     currentPosition++;
     elem = listArray[currentPosition];
     return true;
  }
}
```

Linked list implementation of List ADT

-잘못 짠 코드 부분 : insert 함수

```
-else
{
  tail = addedNode;
  addedNode = tail->next;
  addedNode->next=0;
}
```

-무슨 생각으로? 마지막 노드에 새 노드 주소를 알려주고, 그 다음에 새 노드에 마지막 자리 주소를 알려주고 마지막으로 새 노드의 next가 null을 가리키게 함.

P170(p15) list의 마지막에 요소를 넣는 것

노드 생성 2. 데이터 삽입 3. 마지막 노드의 link에 새 노드 주소를 적어 줌 4. 새 노드의 link에 null을 적어 줌 5. tail에 새 노드 주소를 적어 줌

여기서 3번과 5번은 순서가 유지되어야 함. 즉 3->5 여야 한다.

빠뜨린 중요 사항. 5. Move the tail/head pointer to the new node.

Linked list implementation of List ADT

```
-void List::insert(ListElementType&elem)
 {
  Link addedNode = new Node;
  assert(addedNode);
  if(head==0)
   {
      head=addedNode;
    }
  else
    {
      tail->next = addedNode;
     }
   tail=addedNode;
    addedNode->next = 0;
 }
```

Linked List

```
•bool List::next(ListElementType& elem)
•{
•    assert(current);
•    if (current->next == 0)
•    {
•        return false;
•    }
```

Linked list implementation of List ADT

-잘못 짠 코드 부분 : insert 함수

```
-else
{
  tail = addedNode;
  addedNode = tail->next;
  addedNode->next=0;
}
```

-무슨 생각으로? 마지막 노드에 새 노드 주소를 알려주고, 그 다음에 새 노드에 마지막 자리 주소를 알려주고 마지막으로 새 노드의 next가 null을 가리키게 함.

P170(p15) list의 마지막에 요소를 넣는 것

노드 생성 2. 데이터 삽입 3. 마지막 노드의 link에 새 노드 주소를 적어 줌 4. 새 노드의 link에 null을 적어 줌 5. tail에 새 노드 주소를 적어 줌

여기서 3번과 5번은 순서가 유지되어야 함. 즉 3->5 여야 한다.

빠뜨린 중요 사항. 5. Move the tail/head pointer to the new node.

Linked list implementation of List ADT

-잘못 짠 코드 부분 : first 함수

```
-else
{
    elem=head->elem;
    current = elem;
    return true
}
```

-무슨 생각으로? 새로운 노드에 헤드의 주소를 추가하고 current가 새로운 노드를 가리킨다고 생각함.

First함수는 첫 번째 데이터를 조회하는 함수.

P174(P19) ADT

bool first(ListElementType & elem);

Postcondition: If the list is empty, none. Otherwise, the variable elem contains the first item in L; the “next” item to be returned is the second in L.

-> 즉 **이 함수는 첫 번째 요소를 가져와야 한다는 뜻.**

매개변수의 역할도 생각하자.

매개변수가 참조이므로 초기화하진 않고 주소만 가져온 상태이다.

Linked list implementation of List ADT

```
-bool first(ListElementType &elem)
{
  //After calling first, current points to first item in list.
else
   {
      elem = head->elem;
            //왜?!!?
      current = head;
      return true;
   }
```

Linked list implementation of List ADT

List.h- private:

```
struct Node;
typedef Node* Link;
        struct Node{
        ListElementType elem;
        Link next;
        }
        Link head; // Link current; // Link tail;
```

이렇게 되어 있기 때문에

elem = head->elem;

head->elem : head가 가리키는 주소에 가서 거기에 있는 데이터의 값을 elem에 집어 넣음

Linked list implementation of List ADT

- else

```
{
    elem = head->elem;
        //왜?!!?
    current = head;
    return true;
}
```

여기서 elem=head->elem; 생략을 하니 출력 시 쓰레기 값(-858993460)이 나옴. 왜냐면 초기화를 안 해줘서.

그래서 초기화를 elem=current->elem; 이렇게 하면 예외가 발생됨. 왜냐면 current가 nullptr이어서 가리키는 node가 아예 없었는데 가리키지도 않는 노드의 요소를 불러오라고 하니까 예외가 throw됨.

Current는 first함수를 부르기 전까지 아직 사용이 한번도 안됐고 head는 사용이 되었다. Insert함수를 실행할 때.

그래서 초기화를 Head가 가리키는 것으로 해줘야 함.

Linked list implementation of List ADT

Head에 추가하는 insert를 구현해보기.

예제 요구사항 : Node 공간 할당하고, 데이터를 넣고, 맨 처음 데이터를 넣는 경우는 특별하게 처리함.

(addedNode가 처음 들어가므로 addedNode->next=NULL이 되어야 한다. 그리고 tail이 처음 노드를 가리키게 한다.)

그런 경우가 아니면 새 Node->next=NULL을 가리키게 하고 끝으로 head=addedNode가 되게 한다.

나의 코드 ..

```
void List::insert(const ListElementType&
elem)
{
  Link addedNode = new Node;
  assert(addedNode);
  if(addedNode->elem==NULL)
  {
    addedNode->next=NULL;
    tail=addedNode;
  else
  {
    addedNode->next=head;
  }
  head = addedNode;
```

정답

```
void List::insert(const ListElementType& elem)
{
  Link addedNode = new Node;
  assert(addedNode);
  addedNode->elem=elem;
  if(head==0)
  {
    tail=addedNode;
}

addedNode->next=head;
head= addedNode;
}
```

Inorder list implemented with dummy head node